

```
In [ ]: import os

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from nilearn.image import load_img
from nilearn.glm.second_level import SecondLevelModel
from nilearn.glm import threshold_stats_img
from nilearn.reporting import get_clusters_table

from nilearn.plotting import plot_img_on_surf, plot_stat_map, view_img

import nilearn
print(f"nilearn version: {nilearn.__version__}")
```

nilearn version: 0.13.1

Load 1st level beta maps

```
In [ ]: N_SUBS = 13
CONTRAST = "successful_stop_vs_go"
```

```
In [ ]: current_dir = os.getcwd()
save_dir = f"{current_dir}/output_level2"
os.makedirs(save_dir, exist_ok=True)

beta_map_dir = f"{current_dir}/output"

subs = [str(i).zfill(2) for i in range(1, 16)]
subs.remove("08")
subs.remove("11")
assert len(subs) == N_SUBS
```

```
In [ ]: beta_maps = []
for sub in subs:
    beta_maps.append(load_img(f"{beta_map_dir}/sub-{sub}/sub-{sub}_contrast-

print(f"Loaded beta maps for {len(beta_maps)} subjects.")
```

Loaded beta maps for 13 subjects.

```
In [ ]: # make sure all beta maps same shape and affine
shape1, affine1 = beta_maps[0].shape, beta_maps[0].affine
for i, bm in enumerate(beta_maps):
    if bm.shape != shape1 or not np.allclose(bm.affine, affine1):
        print(f"Warning: beta map for sub-{subs[i]} has different shape or a
        print(f"  shape: {bm.shape} vs {shape1}")
        print(f"  affine:\n{bm.affine}\nvs\n{affine1}")
```

Run 2nd level GLM

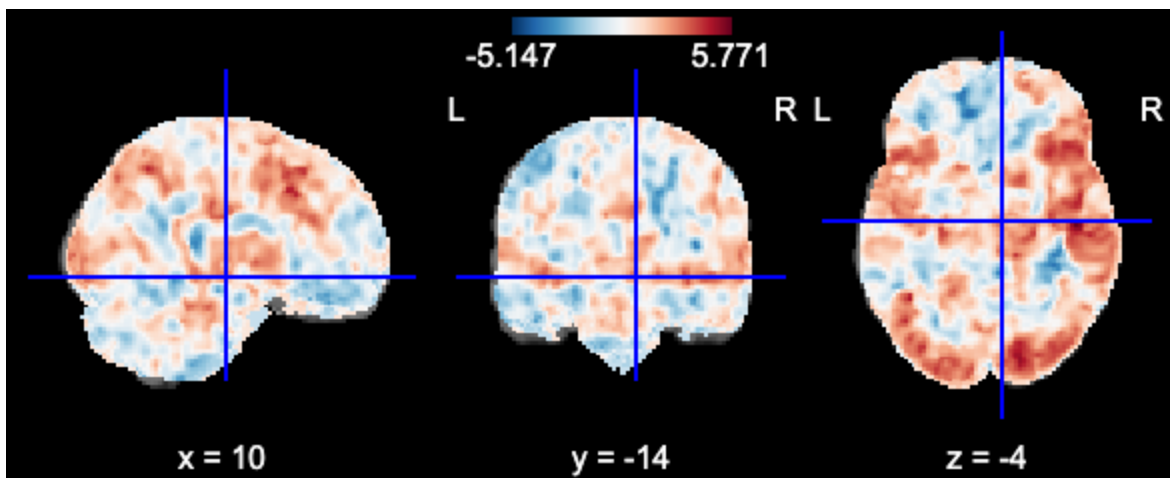
```
In [ ]: design_matrix = pd.DataFrame([1] * len(beta_maps), columns=["intercept"])

second_level_model = SecondLevelModel(smoothing_fwhm=None, n_jobs=-1)
second_level_model.fit(
    beta_maps,
    design_matrix=design_matrix,
)
zmap = second_level_model.compute_contrast(
    second_level_contrast="intercept",
    output_type="z_score",
)
zmap.to_filename(f"{save_dir}/group_contrast-{CONTRAST}_zmap.nii.gz")
```

Plot unthresholded

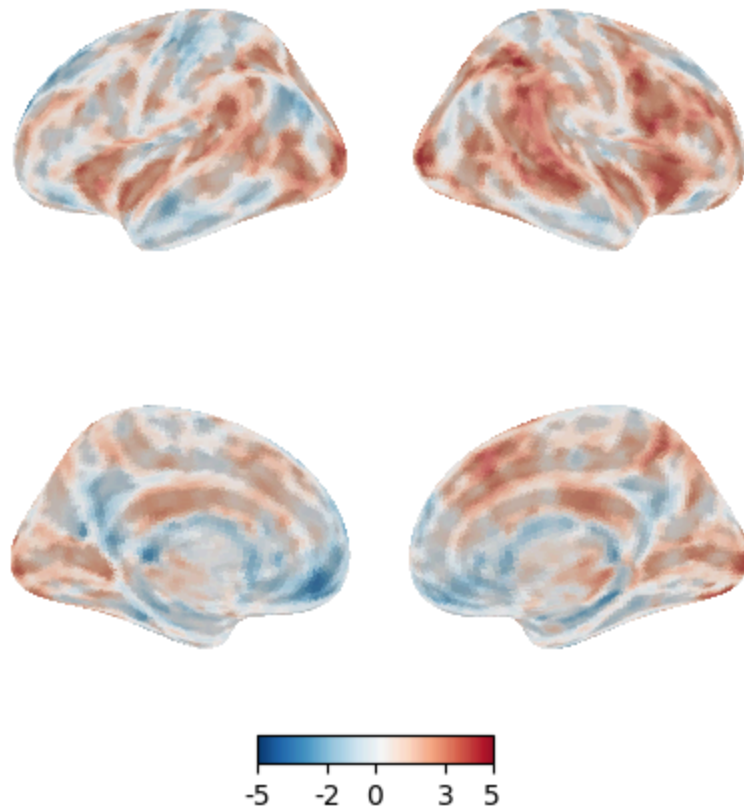
```
In [ ]: view_img(zmap, symmetric_cmap=False, cut_coords=(10, -14, -4))
```

/Users/gabe/.cache/uv/archive-v0/CC2FIMkECm4HZzfQEANhZ/lib/python3.11/site-packages/numpy/_core/fromnumeric.py:840: UserWarning: Warning: 'partition' will ignore the 'mask' of the MaskedArray.
a.partition(kth, axis=axis, kind=kind, order=order)



```
In [ ]: plot_img_on_surf(zmap,
    surf_mesh='fsaverage5',
    views=['lateral', 'medial'],
    hemispheres=['left', 'right'],
    colorbar=True,
    symmetric_cbar=False,
    inflate=True,
    bg_on_data=True,
    title="Successful Stop vs Go 2nd Level Z-map unthresholded")
plt.savefig(f"{save_dir}/group_contrast-{CONTRAST}_zmap_unthresholded.png",
plt.show())
```

Successful Stop vs Go 2nd Level Z-map unthresholded

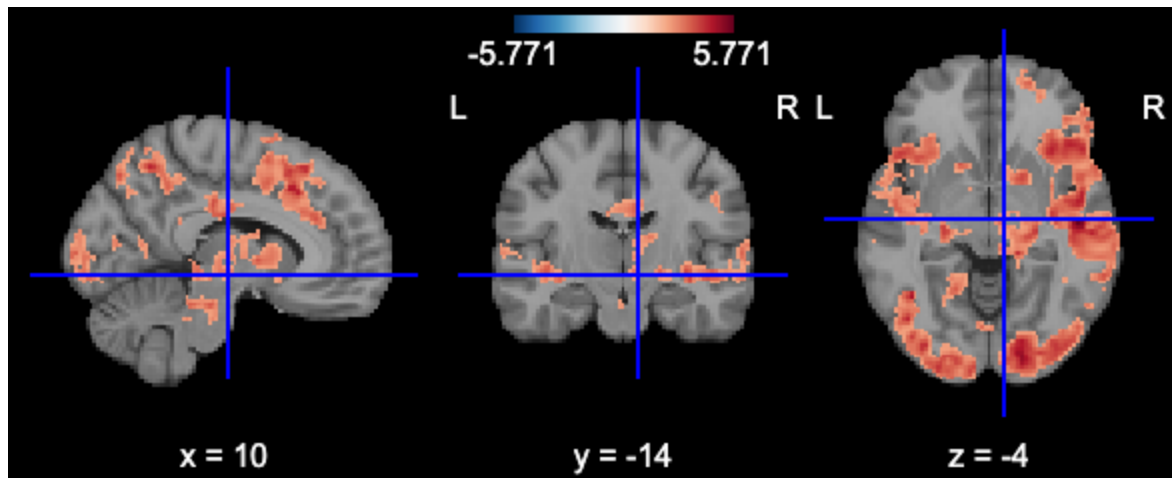


Threshold and plot

Used $z > 2.3$ like in the paper, but chose an arbitrary cluster size (> 30) since the GRFT cluster-thresholding they used wasn't readily available in Nilearn

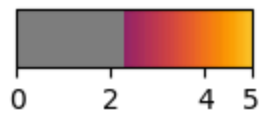
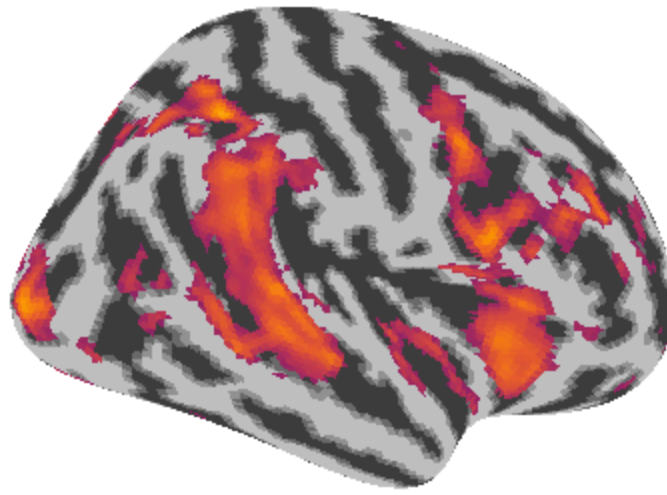
```
In [ ]: thresholded_map, threshold = threshold_stats_img(
    zmap,
    alpha=0.05,                #  $p < 0.05$ 
    height_control=None,        # voxel-level control - if None, use thresholding
    cluster_threshold=30,       # min cluster size - arbitrary... since GRFT
    two_sided=False,           # one-sided test ( $z > 2.3$ )
    threshold=2.3              # voxel-wise  $z$  threshold
)
view_img(thresholded_map, cut_coords=(10, -14, -4))
```

```
/Users/gabe/.cache/uv/archive-v0/CC2FIMkECm4HZzfQEANhZ/lib/python3.11/site-packages/numpy/_core/fromnumeric.py:840: UserWarning: Warning: 'partition' will ignore the 'mask' of the MaskedArray.
  a.partition(kth, axis=axis, kind=kind, order=order)
```

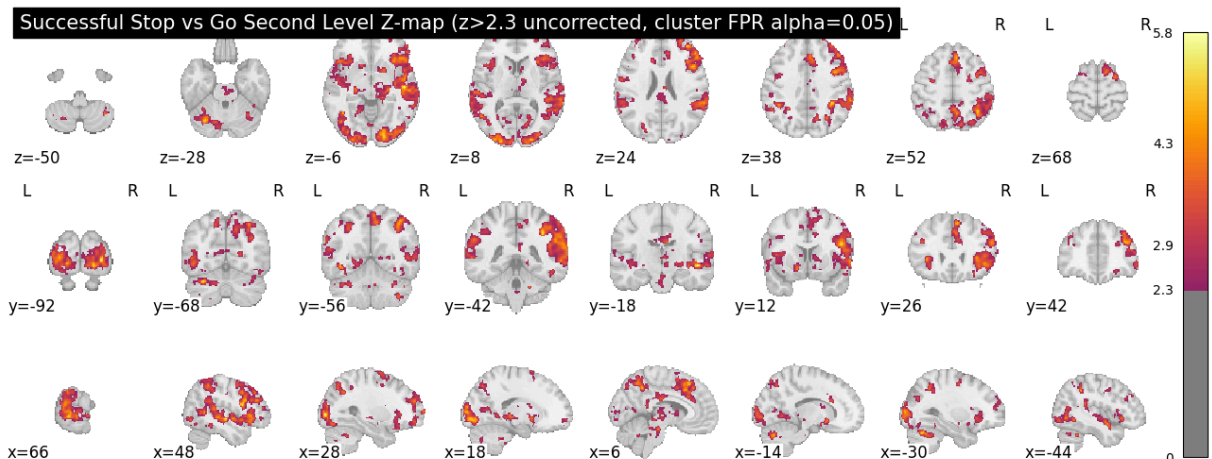


```
In [ ]: # surface plot - sagittal right view like in the paper
plot_img_on_surf(thresholded_map,
                  surf_mesh='fsaverage5',
                  views=['lateral'],
                  hemispheres=['right'],
                  colorbar=True,
                  cmap='inferno',
                  symmetric_cbar=False,
                  threshold=float(threshold),
                  inflate=True,
                  bg_on_data=False,
                  title="Successful Stop vs Go Second Level Z-map thresholded")
plt.savefig(f"{save_dir}/group_contrast-{CONTRAST}_zmap_thresholded.png", dpi=300)
plt.show()
```

Successful Stop vs Go Second Level Z-map thresholded



```
In [ ]: # stat map mosaic to pinpoint coordinates
plot_stat_map(thresholded_map, symmetric_cbar=False, threshold=float(thresho
              draw_cross=False, annotate=True, cmap='inferno',
              display_mode='mosaic', colorbar=True, cut_coords=8,
              title=f"Successful Stop vs Go Second Level Z-map (z>2.3
plt.savefig(f"{save_dir}/group_contrast-{CONTRAST}_zmap_thresholded_mosaic.p
plt.show()
```

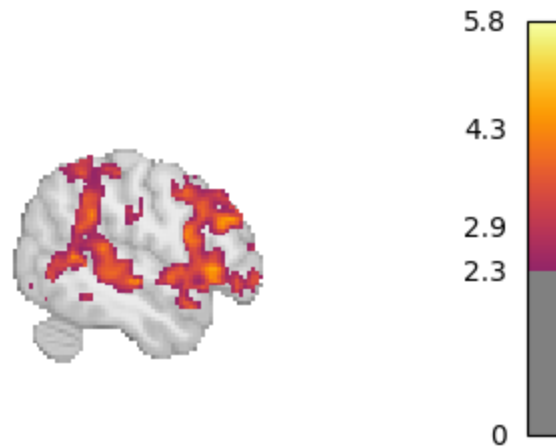


```
In [ ]: # sagittal right
plot_stat_map(thresholded_map, symmetric_cbar=False, threshold=float(thresho
              draw_cross=False, annotate=False, cmap='inferno',
```

```

        display_mode='x', colorbar=True, cut_coords=(52,)),
        # title=f"Successful Stop vs Go Second Level Z-map (z>2.
    )
plt.savefig(f"{save_dir}/group_contrast-{CONTRAST}_zmap_thresholded_x_saggit
plt.show()

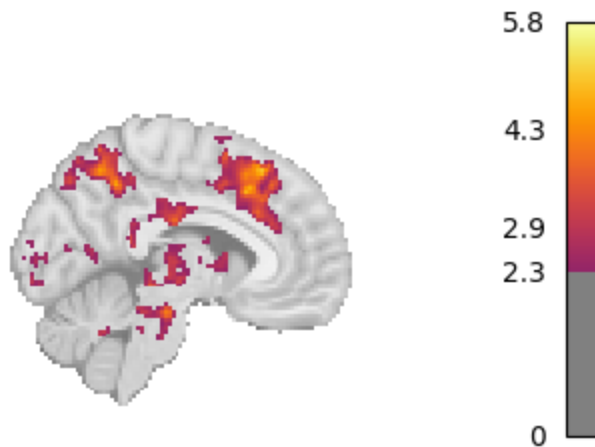
```



```

In [ ]: # sagittal medial
plot_stat_map(thresholded_map, symmetric_cbar=False, threshold=float(thresho
        draw_cross=False, annotate=False, cmap='inferno',
        display_mode='x', colorbar=True, cut_coords=(6,)),
        # title=f"Successful Stop vs Go Second Level Z-map (z>2.
    )
plt.savefig(f"{save_dir}/group_contrast-{CONTRAST}_zmap_thresholded_sagittal
plt.show()

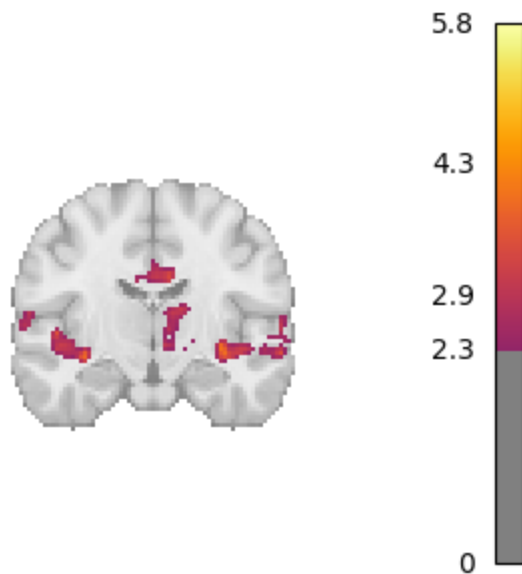
```



```

In [ ]: # coronal
plot_stat_map(thresholded_map, symmetric_cbar=False, threshold=float(thresho
        draw_cross=False, annotate=False, cmap='inferno',
        display_mode='y', colorbar=True, cut_coords=(-14,)),
        # title=f"Successful Stop vs Go Second Level Z-map (z>2.
    )
plt.savefig(f"{save_dir}/group_contrast-{CONTRAST}_zmap_thresholded_y_corona
plt.show()

```



Writeup

I first ran `fMRIPrep` on subs 01 and 02. The bash script can be found in the `preproc` folder, along with PDFs of the HTML QC output.

For the 1st level GLM, I ran this individually for each subject using `Nilearn`. Based on the methods described in the paper, I first gathered key hyperparameters: $TR=2s$, smoothing $FWHM=5$, and a high-pass filter $= 1/66$. I tried to figure out which specific nuisance regressors they used in their design matrix, but this was a bit unclear. I chose to include the 6 basic motion confounds and the default cosine drift model implemented in the `FirstLevelModel` function. To get the events df in the right format, I first had to first remove the first row ($duration=NaN$) and remove spaces from the `trial_type` condition names. An example design matrix used is plotted in the `marimo_glm.py` notebook and its accompanying PDF. I then loaded the events, fMRI BOLD images, and confounds, and fit the GLM. Note, run-03 was missing for sub-12, so I excluded that run and ran the analysis for sub-12 on the remaining 2 runs.

For each subject, I computed the desired contrast of `successful_stop > go` (`output="effect_size"` to get betas), and saved the beta map. I then took these beta maps, 1 for each subject, and used the `SecondLevelModel` function, where I estimated the group-level effect via the `intercept` contrast, yielding a group-level Z-map.

The cluster-wise thresholding they used (GRFT) was not readily available in `Nilearn`, so I instead applied a voxel-wise threshold of `z>2.3` and set an arbitrary cluster size of `> 30 voxels`. I plotted both surface plots and `stat_map` plots, similar to the paper. These plots can be visualized in the exported PDF of this marimo notebook.

I found similar results as reported in the paper (fig. 2c): significant activation for successful stops (greater than go) in the right STN, right pre-SMA, IFC, right Globus Pallidus (GP), right parietal cortex, and right insula.